LESSON 06 · CI / CD · 50 MIN

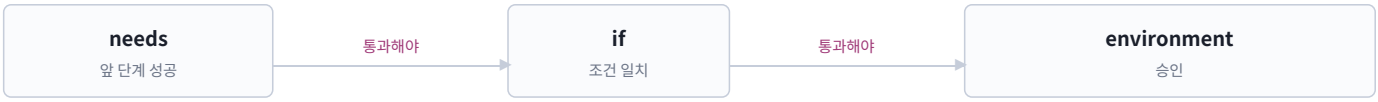
6교시 CD와 통합 — 통과한 것만 내보내기

배포가 안 도는 이유는 세 가지뿐입니다. 앞 단계가 실패했거나, 조건에 맞지 않거나, 승인을 기다리는 중이거나. 셋을 각각 만들어 보고, 시크릿을 안전하게 다루는 법까지 익힙니다.

개념 12분 · 실습 25분 · 마무리 정리 13분

OVERVIEW

배포로 가는 세 개의 관문



셋 중 하나라도 막히면 deploy job 은 **실패가 아니라 '건너뜀'** 으로 표시됩니다. Actions 화면에서 회색으로 보이는 job 은 고장난 것이 아니라 시작되지 않은 것입니다.

REFERENCE

이 시간에 쓰는 명령

명령	하는 일	언제 쓰나
`needs: [test]`	앞 job 이 성공해야 시작	순서와 의존을 만든다
`if: startsWith(github.ref, 'refs/tags/')`	태그일 때만 실행	아무 push마다 배포되는 사고 방지
`environment:`	승인·시크릿을 묶는 단위	운영 배포 앞의 사람 확인
`secrets.NAME`	저장소 밖에 보관된 값	키·토큰은 반드시 이쪽으로
`permissions:`	GITHUB_TOKEN 에 줄 권한	최소한만 열어 준다
`workflow_dispatch`	손으로 실행하는 버튼	재배포·롤백에 유용
`concurrency`	같은 대상 중복 배포 방지	배포가 겹치지 않게
`docker compose / Pages / 레지스트리`	배포 대상은 상황마다 다름	오늘은 Pages 로 실습

FILES

실습용 파일

`.github/workflows/deploy.yml`

```yaml
name: deploy

on:
  push:
    tags: ["v*"]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -r requirements.txt
      - run: pytest --cov=src --cov-report=html
      - uses: actions/upload-artifact@v4
        with:
          name: coverage
          path: htmlcov/

  deploy:
    needs: test
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: coverage
          path: htmlcov
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v3
        with:
```

```
          path: htmlcov
      - id: deployment
        uses: actions/deploy-pages@v4
```

HANDS-ON

실습 순서

STEP 01 Pages 배포 준비

배포 대상을 켭니다.

```
# GitHub 저장소 → Settings → Pages
# Source 를 'GitHub Actions' 로 변경

# 위 '실습용 파일'의 deploy.yml 을 .github/workflows/ 에 만듭니다
git add . && git commit -m "배포 워크플로 추가" && git push
```

확인 포인트

- [ ] Pages 의 Source 가 GitHub Actions 로 되어 있다
- [ ] push 했지만 아직 deploy 는 돌지 않는다 (태그가 아니므로)

> 오늘 배포할 것은 **4교시에서 만든 커버리지 리포트**입니다. 별도 계정이나 결제 없이 실습할 수 있고, 결과물이 브라우저에서 바로 보여 확인하기 좋습니다.

STEP 02 needs 로 순서 만들기 — 오늘의 핵심 (1)

앞 단계가 실패하면 뒤가 어떻게 되는지 확인합니다.

```
# 테스트를 일부러 실패시킵니다
assert cart.total() == 1
git commit -am "실패 유도"
git tag v0.0.1 && git push && git push origin v0.0.1

# Actions 에서 실행 결과를 봅니다
```

확인 포인트

- [ ] test job 은 빨간 X 다
- [ ] deploy job 은 빨간 X 가 아니라 회색이다
- [ ] deploy 를 눌러 보면 skipped 라고 나온다

> **실패와 건너뜀은 다릅니다.** deploy 는 고장난 것이 아니라 시작조차 하지 않았습니다. "배포 job 이 왜 회색이죠"의 답은 대부분 앞 단계입니다.

STEP 03 if 조건 확인하기 — 오늘의 핵심 (2)

태그가 아니면 배포가 안 되는 것을 확인합니다.

```
# 테스트를 원래대로 고치고
git commit -am "수정" && git push

# main 에 push 만 했을 때 → deploy 실행 여부 확인

# 이제 태그를 붙여 봅니다
git tag v0.1.0
git push origin v0.1.0
```

확인 포인트

- [ ] main push 만으로는 deploy 워크플로가 아예 안 뜬다 (on: tags 이므로)
- [ ] 태그를 push 하면 test → deploy 순서로 실행된다
- [ ] 완료 후 Pages 주소로 커버리지 리포트가 열린다

> 아무 커밋마다 운영에 나가면 사고가 납니다. **태그를 붙이는 행위 자체가 "이걸 내보내겠다"는 선언**이 되도록 조건을 걸어 두는 것이 기본입니다.

STEP 04 승인 단계 추가하기 — 오늘의 핵심 (3)

사람이 확인해야 나가도록 만듭니다.

```
# GitHub → Settings → Environments → github-pages
# Required reviewers 를 켜고 본인(또는 짝)을 지정

git tag v0.2.0 && git push origin v0.2.0

# Actions 에서 deploy job 상태 확인 → Review deployments → Approve
```

확인 포인트

- [ ] deploy 가 Waiting 상태로 멈춘다
- [ ] 승인 버튼을 누르면 그때서야 진행된다
- [ ] 누가 언제 승인했는지 기록이 남는다

> 자동화와 통제는 반대말이 아닙니다. **기계가 준비를 다 해 두고 사람은 판단만** 하는 것이 이 단계의 목적입니다.

STEP 05 시크릿 다루기

키를 안전하게 넘기고 로그를 확인합니다.

```
# Settings → Secrets and variables → Actions → New repository secret
# 이름: DEPLOY_TOKEN / 값: 아무 문자열 (예: sk_test_12345)

# 워크플로에 스텝 추가
      - name: 시크릿 확인
        env:
          TOKEN: ${{ secrets.DEPLOY_TOKEN }}
        run: |
          echo "토큰 길이: ${#TOKEN}"
          echo "직접 출력: $TOKEN"

git commit -am "시크릿 실습" && git push
```

확인 포인트

- [ ] 길이는 정상 출력된다
- [ ] 직접 출력한 값은 *** 로 가려진다
- [ ] 코드 어디에도 실제 값이 없다

> **마스킹은 값이 그대로 나올 때만 작동합니다.** base64로 인코딩하거나 한 글자씩 나눠 찍으면 그대로 노출됩니다.
> 그리고 **fork 에서 온 PR 에는 시크릿이 전달되지 않습니다.** 외부 기여자의 PR에서 배포가 실패한다면 이것이 원인입니다.

STEP 06 수동 실행과 롤백

문제가 생겼을 때 되돌리는 방법을 확인합니다.

```
# on: 에 workflow_dispatch 가 있으므로
# Actions → deploy → Run workflow → 브랜치/태그 선택 → 실행

# 롤백 = 직전 정상 버전을 다시 배포
git tag -l
# Actions 에서 v0.1.0 을 선택해 다시 실행
```

확인 포인트

- [ ] Run workflow 버튼이 보인다
- [ ] 이전 태그를 골라 다시 배포할 수 있다

> **롤백은 특별한 기능이 아니라 "예전 것을 다시 배포하는 것"입니다.** 그래서 배포 단위에 태그를 붙여 두는 것이 중요합니다. 되돌릴 지점이 이름으로 남아 있어야 하니까요.

STEP 07 권한 최소화 점검

필요한 만큼만 열어 둡니다.

```
# 워크플로 상단
permissions:
  contents: read
  pages: write
  id-token: write

# Settings → Actions → General → Workflow permissions 도 확인
```

확인 포인트

- [ ] contents 가 read 로 되어 있다
- [ ] 필요 없는 권한이 열려 있지 않다

> permissions 를 아예 안 적으면 저장소 기본값이 적용되는데, 그 기본값이 넓게 열려 있는 경우가 있습니다. **워크플로마다 명시**하는 습관이 안전합니다.

STEP 08 전체 흐름 한 번에 돌려 보기

오늘 배운 것을 이어 붙여 확인합니다.

```
# 코드를 조금 고치고
git commit -am "기능 수정" && git push          # → test 만 실행

git tag v1.0.0 && git push origin v1.0.0        # → test → 승인 → deploy

# Pages 주소에서 갱신된 리포트 확인
```

확인 포인트

- [ ] push 는 검증까지만, 태그는 배포까지 간다
- [ ] 승인 없이는 배포가 나가지 않는다
- [ ] 배포된 리포트가 최신 내용으로 바뀌어 있다

> 이 흐름이 오늘의 결론입니다. **기록(로그) → 검증(테스트) → 자동 반복(CI) → 통제된 배포(CD)**.

TROUBLESHOOTING

자주 만나는 오류

증상	원인	해결
deploy 가 회색으로 건너뛰어짐	needs 실패 또는 if 조건 불일치	앞 job 상태 확인 → on:/if 조건 확인
Resource not accessible by integration	permissions 부족	워크플로에 필요한 permissions 명시
Pages 배포가 404	Settings → Pages 의 Source 미설정	Source 를 GitHub Actions 로 변경
fork PR 에서 시크릿이 비어 있음	보안상 전달되지 않음	정상 동작 · 배포는 태그/메인 기준으로만 실행
태그를 push 했는데 안 돎	태그만 별도로 push 해야 함	git push origin <태그이름> 또는 git push --tags

REVIEW

복습 체크리스트 — 답할 수 있으면 통과

- [ ] deploy job 이 회색으로 표시될 때 확인할 세 가지는?
- [ ] needs 와 if 는 각각 무엇을 막나요?
- [ ] 시크릿을 echo 하면 왜 *** 로 보이고, 그럼에도 안전하지 않은 경우는?
- [ ] 롤백은 기술적으로 무엇을 하는 것인가요?
- [ ] 배포 조건을 '태그일 때만' 으로 두는 이유는?

한 줄 정리

배포는 통과한 것에만 열리는 문입니다. needs 로 순서를, if 로 대상을, environment 로 사람의 판단을 지킵니다.

Logging	레벨 두 관문 · 로거 계층과 propagate · dictConfig 분리 · 예외 기록 · 로테이션
pytest	이름 기반 수집 · 실패 리포트 읽기 · fixture와 scope · parametrize · mock · 커버리지
CI/CD	트리거와 matrix · 캐시와 아티팩트 · needs/if/environment 세 관문 · 시크릿
이어서 공부하면 좋은 것	구조화 로깅(JSON)과 로그 수집 · 통합 테스트와 테스트 컨테이너 · 배포 전략(블루그린·카나리) · 릴리즈 자동화

WORKSHEET

막힌 STEP 번호 · 화면에 나온 오류 메시지 · 물어볼 것